



ESTRUCTURA DE DATOS

Taller 01

II Semestre – 2026
ITI – ICCI



Docentes: Bastián Ruiz – Juan Nilo

Ayudantes: Martín Droguett – Daniel Durán

Fecha del enunciado: 24/08/2026 – 13:00

Fecha entrega taller: 13/09/2026 – 18:00

Desarrollar un sistema básico de gestión de pacientes de un hospital utilizando punteros, aritmética de punteros, programación orientada a objetos, herencia, listas enlazadas, pilas y colas. El sistema deberá recibir los pacientes desde un archivo de texto, procesarlos respetando su orden de llegada y almacenarlos en el servicio correspondiente.

Contexto

El hospital Marmaja, es un recinto de salud en el cual operan diversos departamentos, actualmente su flujo de pacientes es alto, y hay una larga cola de pacientes esperando atención, es por esto se requiere atender a los pacientes de forma debido derivándolos al departamento que necesitan, se cuenta con un conjunto fijo de servicios/departamentos:

- Urgencias
- Medicina General
- Cardiología
- Neurología
- Traumatología
- Cirugía
- Pediatría
- Hospitalización

Los servicios serán los mismos que se utilizarán durante los tres talleres.

Cada paciente posee un identificador, nombre, edad y servicio al que debe ser enviado.

El flujo principal será:

Archivo → Atención → Derivación → Guardado de historial

Requerimientos

Carga de pacientes

El programa deberá leer un archivo de texto que contenga los pacientes. Cada línea tendrá el siguiente formato:

```
ID;Nombre;Edad;Servicio
```

Los pacientes deberán ser incorporados a una Cola en el mismo orden en que aparecen en el archivo. La Cola deberá ser implementada manualmente utilizando nodos y punteros.

Atención de pacientes

Los pacientes deberán ser atendidos siguiendo FIFO. Al atender un paciente:

1. Se retira de la Queue.
2. Se identifica el servicio al que debe ser enviado.
3. Se incorpora al servicio correspondiente.
4. Se registra la atención en el historial de atenciones.

Lista enlazada de servicios

El hospital deberá representarse mediante una lista enlazada principal. Cada nodo de esta lista representará un servicio. Cada servicio deberá contener una segunda lista enlazada con los pacientes que actualmente pertenecen a dicho servicio.

Estructura esperada:

```
Hospital
|
+-- Urgencias -----> Paciente -> Paciente -> ...
|
+-- Medicina General -> Paciente -> Paciente -> ...
|
+-- Cardiologia -----> Paciente -> Paciente -> ...
|
+-- ...
```

Las listas deberán ser implementadas manualmente mediante nodos y punteros. No se permitirá utilizar `std::list`, `std::vector`, `std::queue`, `std::stack` u otras estructuras de la STL para reemplazar las estructuras solicitadas.

Historial de atención

El hospital debe poseer un historial de atenciones implementado mediante una Stack. Cada vez que el paciente sea atendido, deberá registrarse un nuevo evento en el historial. Como mínimo, el historial deberá permitir:

- Agregar una atención.
- Mostrar el historial.

Programación orientada a objetos

El sistema deberá estar desarrollado utilizando POO. Se deberá implementar una jerarquía de clases que incluya herencia y que tenga una justificación dentro del sistema. La jerarquía queda a criterio del grupo, pero deberá existir al menos una clase base y una clase derivada.

Punteros y aritmética de punteros

Las estructuras de datos deberán utilizar memoria dinámica y punteros. Además, se deberá utilizar aritmética de punteros en al menos una operación significativa del programa. No será válido utilizar aritmética de punteros únicamente de forma decorativa o sin relación con el funcionamiento del sistema.

Operaciones mínimas

El programa deberá permitir como mínimo:

- Cargar pacientes desde archivo.
- Mostrar la cola de pacientes pendientes.
- Atender cierta cantidad de pacientes.
- Mostrar el estado general de los servicios.
- Mostrar el historial de atenciones.
- Finalizar la ejecución liberando correctamente la memoria dinámica.

Ejemplo de Ejecución

Archivo de entrada:

```
001;Juan Perez;25;Cardiologia
002;Maria Soto;67;Urgencias
003;Pedro Rojas;43;Cirugia
004;Ana Torres;12;Pediatria
005;Luis Diaz;31;Traumatologia
```

Al iniciar:

```
----- Opción 1 -----
=== HOSPITAL MARMAJA ===
1. Atender pacientes
2. Ver departamento
3. Revisar historial de atención
4. Salir

Seleccionar opción: 1

=== PACIENTES EN ESPERA ===
1. 001 - Juan Perez
2. 002 - Maria Soto
3. 003 - Pedro Rojas
4. 004 - Ana Torres
5. 005 - Luis Diaz

Indique la cantidad de pacientes a atender: 1

=== ATENDIENDO PACIENTES ===
ID: 001
Nombre: Juan Perez
Edad: 25
Servicio: Cardiologia

Paciente enviado a Cardiologia.

----- Opción 2 -----

=== HOSPITAL MARMAJA ===
1. Atender paciente
2. Ver departamento
3. Revisar historial de atención
4. Salir

Seleccionar opción: 2

=== DEPARTAMENTOS/SERVICIOS ===
```

1. Urgencias
2. Medicina General
3. Cardiología
4. Neurología
5. Traumatología
6. Cirugía
7. Pediatría
8. Hospitalización

Seleccionar opción: 3

=== ESTADO CARDIOLOGÍA ===

Pacientes en el departamento de cardiología: 1

Juan Perez (25)

----- Opción 3 -----

=== HOSPITAL MARMAJA ===

1. Atender paciente
2. Ver departamento
3. Revisar historial de atención
4. Salir

Seleccionar opción: 3

=== HISTORIAL DE ÚLTIMAS ATENCIONES DEL HOSPITAL ===

Nombre: Juan Perez | Edad: 25 | Departamento: Cardiología

----- Opción 4 -----

=== HOSPITAL MARMAJA ===

1. Atender paciente
2. Ver departamento
3. Revisar historial de atención
4. Salir

Seleccionar opción: 4

Hasta luego :D.

Consideraciones

- El taller deberá desarrollarse en C++.
- El taller puede realizarse de forma individual o en parejas.
- No se permitirá utilizar contenedores de la STL para reemplazar las estructuras solicitadas.
- Se deberá utilizar memoria dinámica.
- Se deberán implementar destructores o mecanismos equivalentes para liberar correctamente la memoria.
- El programa deberá controlar archivos inexistentes, líneas inválidas, servicios no válidos, pacientes duplicados y estructuras vacías.
- El código deberá estar separado en archivos `.h` y `.cpp` cuando corresponda.
- Se deberá utilizar GitHub para el desarrollo del proyecto.
- La interfaz será de consola.
- Los nombres, cantidad de pacientes y orden del archivo podrán variar durante la evaluación.
- Se evaluará el funcionamiento de las estructuras y no solamente el resultado final.

Obligatorio – Penalización en caso de incumplimiento:

- El README debe incluir integrantes (nombre, rut, nombre en Github, carrera), instrucciones de compilación y ejecución.
- El repositorio debe mantener un historial de commits coherente durante todo el desarrollo.
- La implementación de las estructuras de datos (Linked List, Queue, Stack), debe realizarse de forma manual.
- El taller debe compilar.

En caso de no cumplir con los puntos anteriores, el taller será evaluado con nota 1.0

Rúbrica

A. Estructuras de datos

(50 pts)

Criterio	Subcriterio	Pts	Descripción
Queue (12 pts)	Implementación manual	5	La cola de pacientes pendientes se implementa mediante nodos y punteros, sin usar <code>std::queue</code> .
Queue (12 pts)	FIFO correcto	4	El orden de atención respeta estrictamente el orden de llegada de los pacientes.
Queue (12 pts)	Integración con el flujo	3	La cola se conecta correctamente con la atención y el envío al servicio correspondiente.
Linked List (20 pts)	Lista principal de servicios	6	El hospital se representa mediante una lista enlazada de servicios, implementada manualmente.
Linked List (20 pts)	Lista de pacientes por servicio	8	Cada servicio mantiene su propia lista enlazada de pacientes asociados.
Linked List (20 pts)	Inserción, búsqueda y recorrido	4	Las listas permiten insertar, buscar y recorrer correctamente sus nodos.
Linked List (20 pts)	Manejo correcto de memoria	2	Los nodos de las listas se gestionan sin fugas ni accesos inválidos.
Stack (12 pts)	Implementación manual	4	El historial de atención se implementa como una pila mediante nodos y punteros.
Stack (12 pts)	Comportamiento LIFO	4	Las atenciones se agregan y consultan respetando estrictamente el orden LIFO.
Stack (12 pts)	Historial correctamente integrado	4	El historial se actualiza de forma consistente cada vez que el paciente es atendido.
Punteros (6 pts)	Uso correcto de punteros	3	Las estructuras dinámicas se manipulan correctamente mediante punteros.
Punteros (6 pts)	Aritmética de punteros aplicada	3	Se utiliza aritmética de punteros en al menos una operación significativa del sistema, no de forma decorativa.

B. Programación orientada a objetos**(25 pts)**

Criterio	Subcriterio	Pts	Descripción
POO	Diseño de clases y encapsulamiento	8	Las entidades del sistema están correctamente modeladas mediante clases y encapsulamiento.
Herencia	Jerarquía de clases	7	Existe al menos una clase base y una clase derivada, con una jerarquía justificada dentro del sistema.
Métodos	Constructores/destructores	5	Uso correcto de métodos, constructores y destructores en las clases del sistema.
Diseño	Separación de responsabilidades	5	Las responsabilidades se encuentran correctamente distribuidas entre las clases del sistema.

C. Funcionamiento**(15 pts)**

Criterio	Subcriterio	Pts	Descripción
Carga de datos	Validación del archivo	4	El archivo de entrada se carga y valida correctamente, controlando líneas inválidas.
Procesamiento	Atención de pacientes	4	Los pacientes son procesados correctamente respetando el flujo Queue → Servicio → Historial.
Consultas	Búsqueda de pacientes	3	El sistema permite buscar y consultar correctamente la información de un paciente.
Robustez	Casos borde y errores	4	Se manejan correctamente archivos inexistentes, servicios no válidos, pacientes duplicados y estructuras vacías.

D. Calidad y documentación**(10 pts)**

Criterio	Subcriterio	Pts	Descripción
Modularidad	Organización del código	3	El código está organizado y separado en archivos <code>.h</code> y <code>.cpp</code> cuando corresponde.

(continuación) D. Calidad y documentación

Criterio	Subcriterio	Pts	Descripción
Documentación	README	2	El README incluye integrantes, instrucciones de compilación y ejecución.
Git	Control de versiones	3	El repositorio presenta un historial de commits coherente en GitHub.
Estilo	Legibilidad y buenas prácticas	2	El código presenta nombres descriptivos, indentación consistente y buenas prácticas generales.

Puntaje total: 100